



POLYTECH[®]
ORLÉANS

École d'Ingénieurs de l'Université d'Orléans

Programmation Initiation à l'algorithmique

Rémy LECONGE



CONFERENCE DES
GRANDES ECOLES



Plan du cours

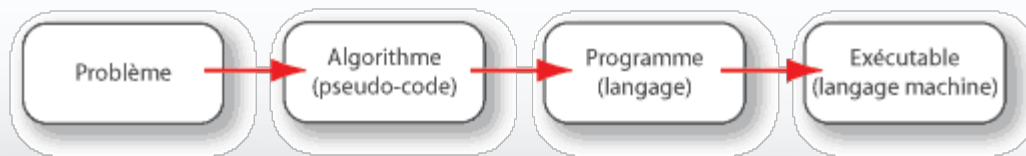
- Introduction
- Les éléments syntaxiques
 - ☐ Variable et constante
 - ☐ Affectation
 - ☐ Les entrées/sorties
 - ☐ Les conditions
 - ☐ Les boucles
 - ☐ Les tableaux
 - ☐ Les procédures et fonctions



Introduction

➤ Pourquoi l'algorithmique ?

- ☐ L'objectif est d'apprendre à la machine comment traiter un « problème »
- ☐ Pour cela, il faut:
 1. On identifie le problème à traiter, en inventoriant les fonctionnalités nécessaires, les tenants et les aboutissants, les règles explicites ou implicites, les cas tordus, etc. C'est l'**analyse**.
 2. On procède à un découpage de l'application entre une procédure qui jouera le rôle de chef d'orchestre, ou de donneur d'ordre, et des modules périphériques (fonctions ou sous-procédures) qui joueront le rôle de sous-traitants spécialisés. C'est l'**analyse fonctionnelle**.
 3. On détaille l'enchaînement logique des traitements de chaque (sous-)procédure ou fonction : c'est l'**algorithmique**.
 4. On procède sur machine à l'écriture (et au test) de chaque module dans le langage voulu : c'est le codage, ou la programmation proprement dite.



Introduction

➤ l'Algorithmique

- ❑ est un travail de programmation à visée «universelle»: un algorithme ne dépend pas :
 - du **langage** dans lequel il est implanté (il est écrit en pseudo-code)
 - ni de la **machine** qui exécutera le programme correspondant
- ❑ s'interroge sur des problèmes importants dans la conception d'applications professionnelles (ce qu'on appelle la complexité algorithmique)
 - En combien de temps un algorithme va-t-il atteindre le résultat escompté?
 - De quel espace a-t-il besoin?
 - ↳ choix du meilleur algorithme pour résoudre un problème en fonction des contraintes.



Les éléments syntaxiques

Ils vont être la base constitutive de tous algorithmes, leur enchainement détaillera les tâches à réaliser pour répondre à la problématique.



Les Variables

➤ déclaration de variable :

une variable va correspondre a un emplacement de la mémoire de l'ordinateur, dont on peut changer la valeur.

Variables <liste des identificateurs> : Type

Un identificateur est le nom que le développeur souhaite attribuer à sa variable
Le type caractérise la valeur stockée en mémoire (réels, entier, booléen, caractère, chaîne de caractères)

Exemple de déclaration de variable:

variables nbVal, val : entiers

ok: chaîne de caractère

X: réels



Les Variables

➤ affecter une valeur dans une variable :

Cette action stocke dans l'emplacement de la mémoire de l'ordinateur associé à la variable une valeur.

<identificateurs> ← <constante> ou <expression> ou <identificateur>

Exemple d'affectation de variable:

variables val, val2 : entiers

val ← 50

val ← val * 2

val2 ← val



Les Constantes

➤ déclaration de constante :

une constante va correspondre à un emplacement de la mémoire de l'ordinateur, mais contrairement à la variable une fois sa valeur fixée on ne peut plus la changer.

```
constante(<identificateur> : type) ← <expression>
```

Exemple de déclaration d'une constante :

Constante (Max: entier) ← 1
(Pi: réel) ← 3.14159



Les entrées / sorties

➤ saisir une donnée

Permet de stocker une donnée fournie au clavier par l'utilisateur dans un emplacement de la mémoire de l'ordinateur.

```
saisir(<liste des identificateurs> )
```

Exemple de saisie de donnée :

```
variables val, val2 : entiers  
x, y : réels
```

```
saisir(val)  
saisir(val2)  
saisir(x, y)
```



Les entrées / sorties

➤ afficher une donnée

Permet d'afficher à l'écran une information, chaîne de caractère, résultat, opérations, contenu d'un emplacement de la mémoire de l'ordinateur, etc...

afficher(<liste des identificateurs>)

Exemple d'affichage de données:

variables val : entiers

saisir(val)

Afficher (val)

Afficher (val, "et le double vaut",val*2)

Afficher ("bonjour, bienvenue dans
l'application ")

Afficher (2+3)



Traitement conditionnel

➤ si ... alors ... sinon

Si l'expression logique (la condition) prend la valeur **vrai**, le premier bloc d'instructions est exécuté; si elle prend la valeur **faux**, le second bloc est exécuté (s'il est présent).

```
Si <expression logique>  
| alors instructions  
| [sinon instructions]  
fsi
```

Exemple pour l'instruction si alors sinon:

```
variables age : entiers  
afficher("saisir votre age :")  
saisir(age)
```

```
si age>=18  
alors afficher ("vous êtes majeur")  
sinon afficher ("vous êtes mineur")  
fsi
```



Traitement conditionnel

➤ si ... alors ... sinon

☐ Autre exemple

```
variables age : entiers  
afficher("saisir votre age :")  
saisir(age)  
  
si age >= 18  
alors afficher ("vous êtes majeur")  
fsi
```



Traitement conditionnel

➤ si ... alors ... sinon

- ❑ Expression logique: elle peut être simple ou composée de plusieurs termes et dans ce cas ceux-ci sont connectés via des opérateurs logiques

¬ non logique

∧ et logique

∨ ou logique

- ❑ Rappel des tables de vérité:

a	¬ a
vrai	faux
faux	vrai

a	b	a ∧ b
vrai	faux	faux
vrai	vrai	vrai
faux	faux	faux
faux	vrai	faux

a	b	a ∨ b
vrai	faux	vrai
vrai	vrai	vrai
faux	faux	faux
faux	vrai	vrai



Traitement conditionnel

➤ si ... alors ... sinon

☐ Instructions conditionnelles imbriquées

variables moyenne : réels

Afficher("saisir le moyenne de l'élève :")

Saisir(moyenne)

si moyenne ≥ 10

alors afficher ("élève admis")

 si moyenne ≥ 12

 alors si moyenne $\geq 12 \wedge$ moyenne < 14

 alors afficher (" mention AB")

 sinon si moyenne $\geq 14 \wedge$ moyenne < 16

 alors afficher ("mention B")

 sinon afficher ("mention TB")

 fsi

 fsi

 sinon afficher ("mention passable")

 fsi

sinon afficher ("élève en échec")

fsi



Traitement conditionnel

➤ La sélection sur choix multiple

S'il y a plus de deux choix possible cette instruction rend l'écriture du traitement conditionnel plus facile

```
Selon <identificateur>  
  (liste de) valeurs : instructions  
  (liste de) valeurs : instructions  
  ...  
  [autre : instructions]
```

Exemple de sélection à choix multiple :

Selon c

'R': afficher ("le feu est rouge")

'O': afficher ("le feu est orange")

'V': afficher ("le feu est vert")

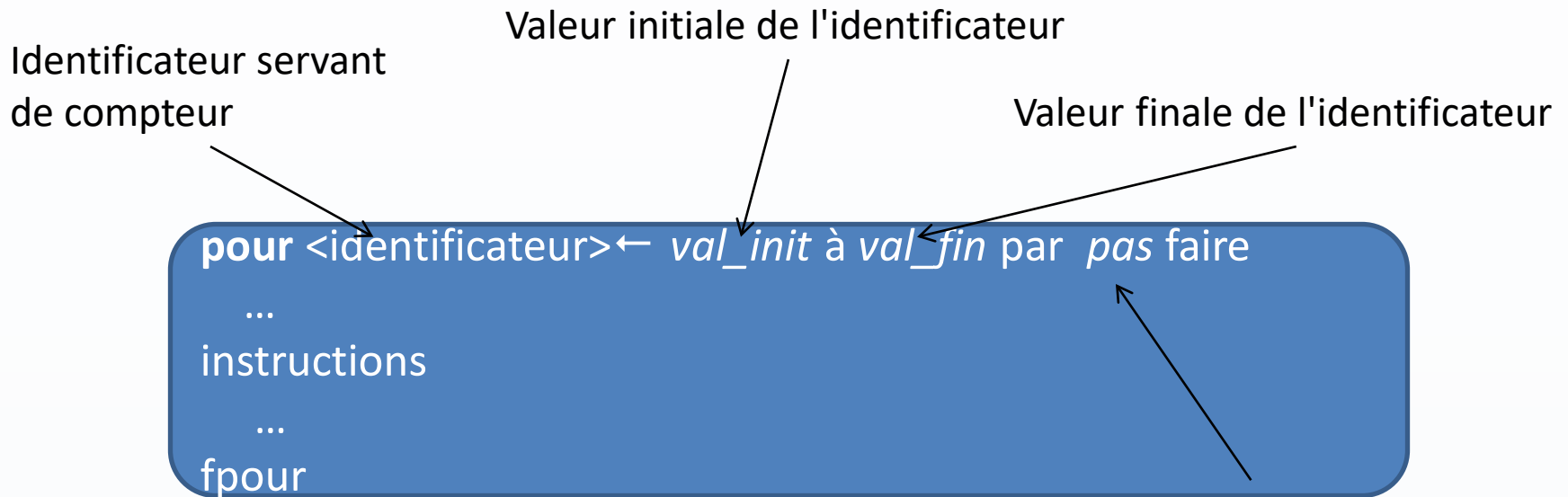
Autre: afficher ("le feu est hors service")



Les boucles

➤ La boucle **pour**, Elle permet de répéter une série d'instruction un certains nombre de fois.

détails de la syntaxe:



Implicitement la boucle « **pour** » réalise:

- Initialisation de l'identificateur (variable) grâce à la valeur initiale
- Incrémente cette variable de la quantité *pas* à chaque itération
- Vérifie que cette variable ne dépasse pas *val_fin*



Les boucles

➤ Exemple boucle pour : (faire la somme de N valeurs)

```
variables N, totalValeurs,cpt, valeur : entiers  
afficher("Combien de valeurs voulez-vous saisir ?")  
saisir(N)  
totalValeurs←0  
pour cpt ←1 à N par pas=1 faire  
  afficher("Donnez une valeur :")  
  saisir(valeur)  
  totalValeurs←totalValeurs+ valeur  
fpour  
afficher("Le total des ", N , "valeurs est " , totalValeurs)
```



Les boucles

➤ La boucle **tant que ... faire**

Elle permet de répéter une série d'instruction tant qu'une condition est vraie (le nombre d'itérations est en conséquence pas nécessairement connu.

Rq: si la condition est fausse alors la boucle n'est jamais exécutée

```
amorçage  
Tant que <expression logique(vraie)> faire  
| instructions  
| ...  
ftq
```

Amorçage: il correspond à l'affectation des variables qui seront par la suite testées dans l'expression logique.

Attention, les variables testées dans l'expression logique doivent **impérativement** être remise à jour à l'intérieur de la boucle (cas provoquant la sortie de boucle), sinon vous provoquez une boucle infinie.



Les boucles

➤ Exemple boucle tant que : (faire la somme de N valeurs, N apriori inconnu)

constante(STOP : entier) $\leftarrow -1$

variables val, totalValeurs: **entiers**

totalValeurs $\leftarrow 0$

afficher("Donnez une valeur,", STOP, " pour finir.")

saisir(val)

tant que val $\neq$ STOP **faire**

totalValeurs $\leftarrow$ totalValeurs + val

afficher("Donnez une autre valeur,", STOP, " pour finir.")

saisir(val)

ftq

afficher("La somme des valeurs saisies est", totalValeurs)



Le boucles

➤ comment faire le choix entre une boucle **pour** et une boucle **tant que** ?

si le nombre d'itérations est connu à l'avance

↳ **choisir la boucle pour**

si la boucle doit s'arrêter quand survient un évènement

↳ **choisir la boucle tant que**



Les boucles

➤ La boucle **répéter ... tant que**

Elle permet de répéter une série d'instruction ou moins une fois et tant que la condition est vraie (le nombre d'itérations est en conséquence pas nécessairement connu).

```
Répéter  
| instructions  
| ....  
Tant que <expression logique(vraie)>
```

Attention, les variables testées dans l'expression logique doivent **impérativement** être mise ou remise à jour à l'intérieur de la boucle (cas provoquant la sortie de boucle), sinon vous provoquez une boucle infinie.



Les boucles

➤ Exemple boucle répéter ... tant que : (calcul du quotient $1/N$)

variables N: entiers

répéter

afficher("Donnez la valeur de N : ")

saisir(N)

tant que $N \neq 0$

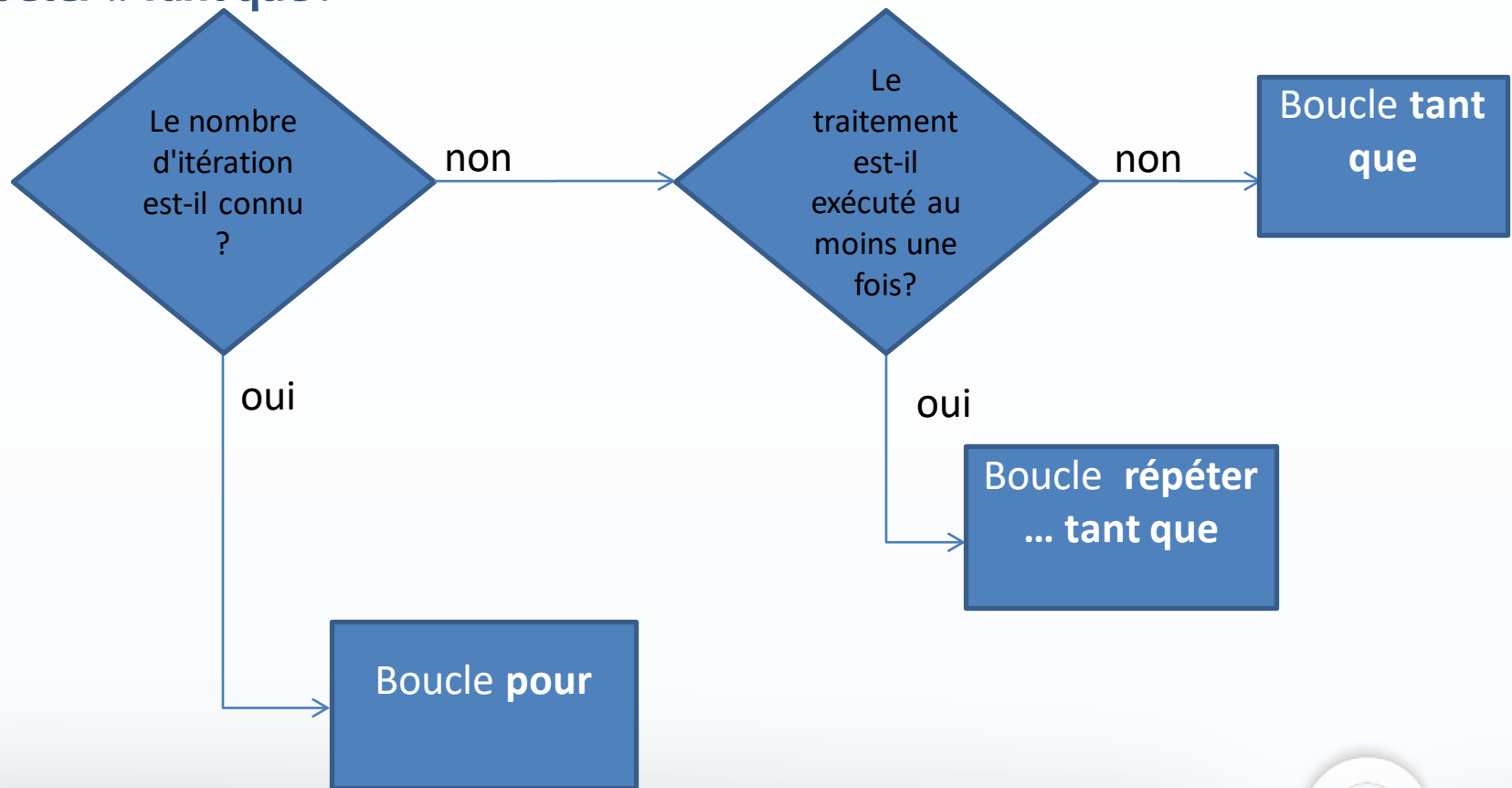
afficher("La valeur $1/N$ vaut", $1/N$)

La boucle est utilisée ici pour empêcher l'utilisateur de saisir 0 (et donc de faire une division par 0) dans ce cas on lui redemande une autre valeur.



Le boucles

- comment faire le choix entre une boucle **pour**, une boucle **tant que** et une boucle **répéter .. Tant que** ?



Les tableaux

➤ déclaration d'un tableau :

Un tableau peut être vu comme une succession d'emplacements de la mémoire de l'ordinateur permettant de stocker des valeurs ayant une cohérence entre elles, et dont on peut changer les valeurs. Les tableaux sont typés, c'est-à-dire que tous les éléments constitutifs d'un tableau ont le même type (entier, réels, chaîne, etc...)

Tableau <identificateur>(taille) : Type

L'identificateur est le nom que le développeur souhaite attribuer à son tableau

Taille correspond au nombre d'éléments constitutifs de tableau (nb emplacements mémoire)

Le type caractérise les valeurs stockées dans **tous** les emplacements mémoire de ce tableau

Exemple de déclaration de tableau :

tableau tabX (10) : entiers

tableau tab_reel(15): réel



Les tableaux

➤ utilisation d'un tableau :

Pour accéder à un des éléments du tableau, on spécifie l'indice de l'emplacement mémoire (case) que l'on souhaite "manipuler" (lire ou écrire)
Attention : il conviendra de définir le standard de numérotation des cases (numérotation commençant soit à 0, soit à 1)

Exemple d'utilisation d'un tableau :

variable i : entier

tableau tabX (10) : entiers

Pour $i \leftarrow 1$ à 10 par pas de 1

tabX(i) $\leftarrow i * 2$

Afficher ("contenu de la case", i, ":", tabX(i))

fpour



Les procédures et les fonctions

➤ structuration du programme

- ❑ un programme est une série d'instructions qui seront réalisées par l'ordinateur
- ❑ De manière à rendre le programme surtout s'il est long, plus lisible plus facile à maintenir, et surtout si un même sous ensemble d'instructions est utilisé plusieurs fois dans le même programme, on structure le programme en plusieurs "parties" ou "entités". Chacune de ces entités correspondant un petit morceau de programme fonctionnel.
- ❑ Informatiquement parlant, cette entité de code est appelé une procédure

➤ procédure vs fonction

- ❑ une procédure est donc un petit morceau de code réalisant un traitement particulier. Réalité la terminologie algorithmique est plus précise que cela.
- ❑ Une procédure est une série d'instructions réalisant un traitement particulier
- ❑ Une fonction est une procédure particulière car elle retourne(fournit) un résultat issue de la réalisation des instructions qui la composent, ce qui n'est pas le cas de la procédure



Les procédures et les fonctions

- structuration du programme
 - un programme commence par une procédure principale

Procédure principale()

Début

Instructions

....

fin

Exemple de procédure principale:

Procédure principale()

Début

variables N: entier

répéter

afficher("Donnez la valeur de N : ")

saisir(N)

tant que N=0

afficher("La valeur 1/N vaut",1/N)

fin



Les procédures et les fonctions

la procédure principale peut appeler des fonctions

➤ Syntaxe de la fonction

```
fonction NomF ([liste paramètre : type]) : type_retour  
Début  
Instructions  
....  
retourne (<resultat> )  
fin
```

NomF est le nom de la fonction, le développeur est libre de saisir le nom de son choix.

Remarque : choisir des noms "parlant", éviter à tout prix fonction1, fonction2, etc...

<résultat> peut être une donnée, une constante, un identificateur de variable

[liste type paramètre] est une liste optionnelle de paramètre



Les procédures et les fonctions

la procédure principale peut appeler d'autres procédures

➤ Syntaxe d'une procédure

```
procédure NomP ([liste paramètre : type])
```

```
Début
```

```
Instructions
```

```
....
```

```
fin
```

NomP est le nom de la procédure, le développeur est libre de saisir le nom de son choix.
Remarque : choisir des noms "parlant", éviter à tout prix procédure1, procédure2, etc...
[liste type paramètre] est une liste optionnelle de paramètre



Les procédures et les fonctions

➤ Exemple

```
Fonction inverse(X: entier par valeur): réel
Debut
Retourne( 1/X)
Fin
Procédure Introduction ()
Debut
Afficher ("Ce programme permet de calculer 1/N" )
Fin
Procédure principale()
Debut
variables N: entiers
Introduction()
    répéter
        afficher("Donnez la valeur de N : ")
        saisir(N)
    tant que N=0
    afficher("La valeur 1/N vaut ", inverse(N))
fin
```

Résultat d'exécution :

Ce programme permet de calculer 1/N
Donnez la valeur de N : 0
Donnez la valeur de N : 2
La valeur 1/N vaut 0.5



Algorigramme

- Un algorithme peut être représenté sous forme d'un schéma, appelé algorigramme.
- Il modélise le déroulement de l'algorithme.
- Il est composé de 2 entités graphiques : des rectangles (représentant les instructions certaines (affectation, saisie, affichage, calcul etc...)) et des losanges (représentant des instructions conditionnelles ou itératives vis-à-vis de leur condition d'arrêt, provoquant un "aiguillage" dans le déroulement de l'algorithme)

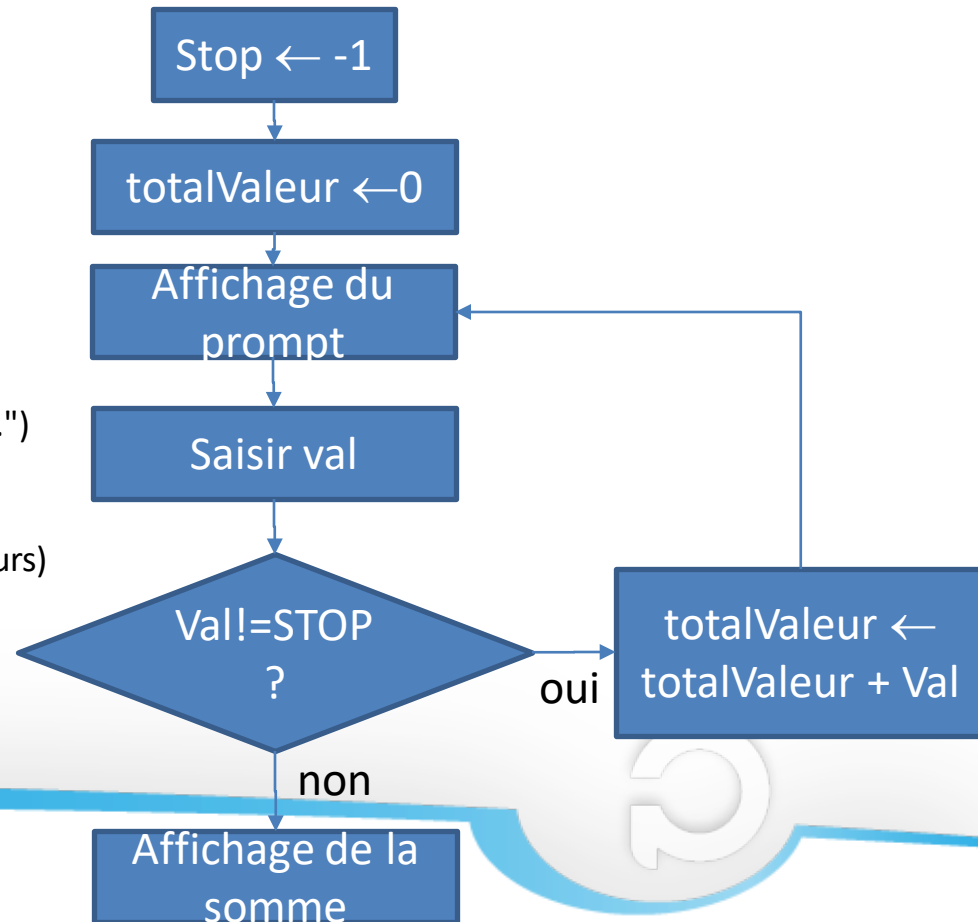


Algorithme

- Exemple : Reprenons l'exemple de la somme de N nombres

```
constante(STOP : entier) ← -1
variables val, totalValeurs: entiers

totalValeurs ← 0
afficher("Donnez une valeur,", STOP, " pour finir.")
saisir(val)
tant que val ≠ STOP faire
    totalValeurs ← totalValeurs + val
    afficher("Donnez une autre valeur,", STOP, " pour finir.")
    saisir(val)
ftq
afficher("La somme des valeurs saisies est", totalValeurs)
```



La complexité algorithmique

- Complexité en temps
- Complexité en espace



La complexité algorithmique

➤ son origine

❑ 1 problème à résoudre ➔ de 1 à N façon(s) de coder (algorithme) sa résolution

❑ Exemple :

```
fonction puissance1(x :réel; N :entier) : réel
debut
variables resultat : réel
i : entier
    resultat ← 1
    pour i de 1 à N par pas de 1 faire
        resultat ← resultat * x
    fpour
retourne (resultat)
fin
```

```
fonction puissance2(x :réel; N :entier) : réel
debut
variables resultat : réel
    Si (N=0) alors
        resultat ← 1
    Sinon
        resultat ← x * puissance2(x, N-1)
    fsi
retourne (resultat)
fin
```

❑ Problème comment choisir parmi ces différents algorithmes?

↳ Création de critères objectifs (mesure) pour aider le développeur à choisir:

- la complexité en temps
- La complexité en espace

❑ La complexité va s'exprimer sous la forme d'une polynôme



La complexité algorithmique

➤ la complexité en temps

Le temps d'exécution de l'algorithme sera évalué en considérant le nombre d'instructions élémentaires devant être exécuter : une instruction est élémentaire est exécutée en un temps fixe

instruction élémentaire :

- ❑ Une instruction peut être déclaré élémentaire en fonction de l'hypothèse initiale:
 - si l'on manipule un entier est stocké sur 4 octets (cas général)
 - ↳ L'addition ou toute autre opération est considérée comme instruction élémentaire
 - si on manipule des entiers de grande taille (jusqu'à plusieurs centaines octets)
 - ↳ L'addition(ou toute autre opération) n'est élémentaire car manipuler un entier revient en définitive à manipuler autant de série de 4 octets qu'il y en a de présent dans celui-ci.



La complexité algorithmique

➤ la complexité en espace

L'espace est le nombre d'octets utilisé par l'exécution de l'algorithme.

➤ Compromis espace-temps

Parfois il n'existe pas de "meilleur" algorithme :

- ❑ Exemple : soit un problème mettant en jeu N valeurs on pourrait trouver :
 - Une solution algorithmique de complexité en temps de n^2 et de complexité en espace constante
 - Une solution algorithmique de complexité en temps de n et de complexité en espace de n

Ces 2 solutions ne sont pas comparables, donc on ne peut pas dire que l'une est meilleure que l'autre.



La complexité algorithmique

➤ Simplification:

Hormis dans le cas d'application temps réel ou embarquée, où l'on cherche une optimisation maximale, et conséquence estimer la complexité en temps en descendant jusqu'au nombre de coup d'horloge du PC pour réaliser une affectation, une addition, etc... Dans tout les autres cas, on définit la complexité à une constante multiplicative près, pour indiquer cette simplification on utilise la notation suivante $O(\dots)$ appelé notation de Landeau.

Par exemple : $O(n)$ ou $O(n^2)$

En conséquence si l'on a 2 algorithmes différents dont la complexité en temps est respectivement $11n$ et $3n$, ces 2 algorithmes auront une complexité en temps en $O(n)$ et l'on considéra qu'il sont équivalent en complexité en temps, l'un n'est pas meilleur que l'autre.

Par contre si on a 2 algorithmes l'un ayant une complexité en temps en $O(n)$ et l'autre en $O(n^2)$ on pourra affirmer que l'algorithme en $O(n)$ est meilleur.

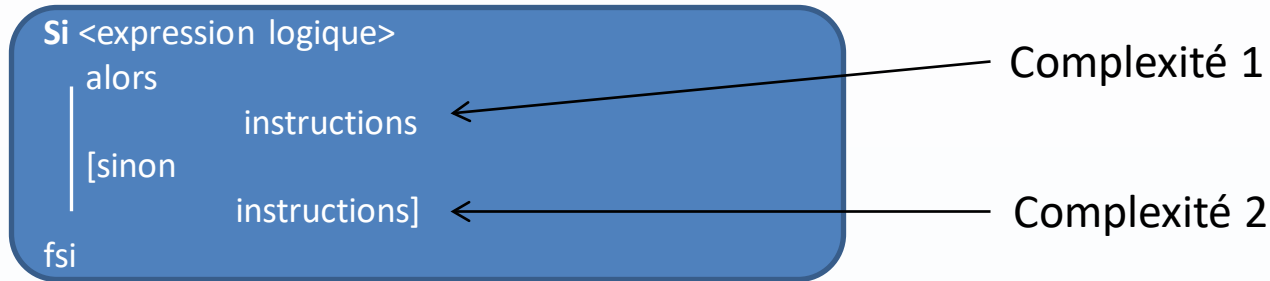


La complexité algorithmique

- De 2 parties d'algorithme:

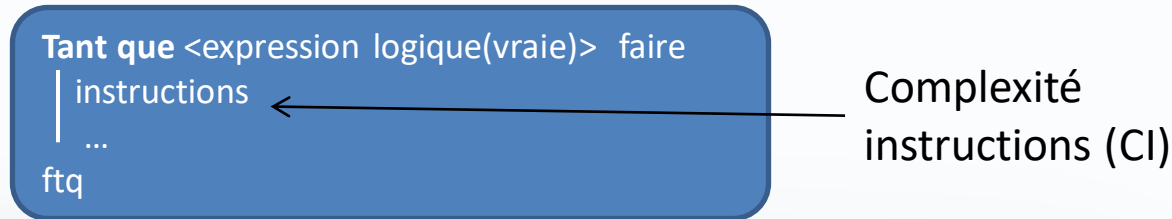
Complexité_{totale} = complexité_{partie1} + complexité_{partie2}

- D'un branchement conditionnel



Complexité = max{ complexité 1, complexité 2 }

- D'une boucle



$$\text{Complexité} = \sum_{k=1}^m CI_k$$



La complexité algorithmique

➤ Exemples d'estimation de la complexité:

Exemple d'une permutation entre deux éléments de tableau

tmp $\leftarrow$ S[i], coût c1 (1)

S[i] $\leftarrow$ S[j], coût c2 (1)

S[j] $\leftarrow$ tmp, coût c3 (1)

Coût total

$$c1 + c2 + c3 \approx O(1)$$

Exemple du calcul de la factorielle d'un entier

fonction factorielle(entier n) : entier

variable i, resultat : entier

début

resultat $\leftarrow$ 1

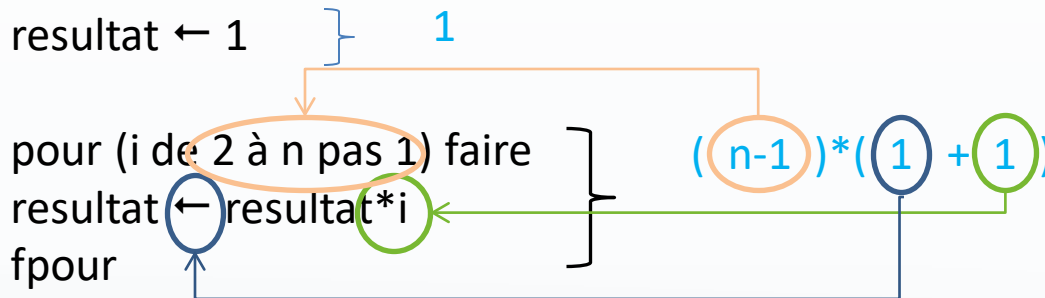
pour (i de 2 à n pas 1) faire

resultat $\leftarrow$ resultat * i

fpour

retourne (resultat)

fin



Coût total

$$1 + (n-1) * 2 \approx O(n)$$



La complexité algorithmique

- La complexité rapportée à une échelle de temps :

complexité flops	$\ln(n)$	n	n^2	2^n
10^6	0,013ms	1s	278 heures	10 000 ans
10^9	0,013 μ s	1ms	15 min	10 ans
10^{12}	0,013ns	1 μ s	1s	1 semaine

Flops (FLoating point Operations Per Second): opérations à virgule flottante par seconde

